

Cliff Walking

Mathematical Walkthrough of the PyCharm Project

`environment.py` `value_iteration.py` `simulation.py`

Reinforcement Learning

May 26, 2026

Outline

- 1 MDP Foundations
- 2 `environment.py`
- 3 `value_iteration.py`
- 4 `simulation.py`
- 5 `main.py`
- 6 Summary

Markov Decision Process

A finite MDP is the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$

Components

- $\mathcal{S} = \{s_0, \dots, s_{|\mathcal{S}|-1}\}$ finite states
- $\mathcal{A} = \{a_0, \dots, a_{|\mathcal{A}|-1}\}$ finite actions
- $\mathcal{T}(s, a, s') = \Pr[S_{t+1}=s' \mid S_t=s, A_t=a]$
- $\mathcal{R}(s, a)$ expected reward
- $\gamma \in [0, 1)$ discount factor

Objective

Find $\pi : \mathcal{S} \rightarrow \mathcal{A}$ maximising the discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Markov property

$$\Pr[S_{t+1} \mid S_t, A_t, \dots, S_0] = \Pr[S_{t+1} \mid S_t, A_t]$$

The future depends *only* on the current state and action – not the full history.

Cliff Walking instance

$ \mathcal{S} $	48 cells
$ \mathcal{A} $	4 directions
γ	0.99

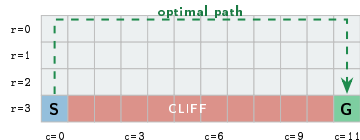
State encoding

Grid: 4 rows $\times$ 12 columns = 48 cells.

$$s = r \cdot 12 + c \in \{0, 1, \dots, 47\}$$

$$(r, c) = (\lfloor s/12 \rfloor, s \bmod 12)$$

- $S_{\text{start}} = 3 \cdot 12 + 0 = 36$
- $S_{\text{goal}} = 3 \cdot 12 + 11 = 47$
- $\mathcal{C} = \{37, 38, \dots, 46\}$ (cliff)



Wall-bounce rule

$$r' = \max(0, \min(3, r + \Delta r))$$

Out-of-bounds moves keep the agent in place.

Action set $\mathcal{A}$

Index	Name	$(\Delta r, \Delta c)$
0	Up	$(-1, 0)$
1	Down	$(+1, 0)$
2	Left	$(0, -1)$
3	Right	$(0, +1)$

```
1 NUM_ROWS, NUM_COLS = 4, 12
2 NUM_STATES = 48 # |S|
3 NUM_ACTIONS = 4 # |A|
4
5 # s = 12*r + c
6 def rc_to_state(row, col):
7     return row * NUM_COLS + col
8
9 # (r, c) = divmod(s, 12)
10 def state_to_rc(s):
11     return divmod(s, NUM_COLS)
12
13 START_STATE = 36 # (3, 0)
14 GOAL_STATE = 47 # (3, 11)
15 CLIFF_STATES = {3*12+c
16                 for c in range(1,11)}
```

Why flat indexing?

The reward matrix and transition tensor are NumPy arrays:

$$\mathcal{R} \in \mathbb{R}^{|S| \times |A|} \quad \mathcal{T} \in \mathbb{R}^{|S| \times |A| \times |S|}$$

Flat integer indices let us write $R[s,a]$ and $T[s,a,s']$ without any tuple conversion at runtime.

Example indices

Cell	(r, c)	s
Start	(3, 0)	36
Goal	(3, 11)	47
Cliff	(3, 1)–(3, 10)	37–46

Definition

Let $s'(s, a)$ be the wall-bounced next cell. Then:

$$\mathcal{R}(s, a) = \begin{cases} -100 & s'(s, a) \in \mathcal{C} \\ 0 & s'(s, a) = S_G \\ -1 & \text{otherwise} \end{cases}$$

Shape: $\mathcal{R} \in \mathbb{R}^{48 \times 4}$

State	$a=0$	$a=1$	$a=2$	$a=3$
$s = 36$ (Start)	-1	-100	-1	-100
$s = 24$ (safe)	-1	-1	-1	-1
$s = 46$	-1	-1	-1	0
$s = 47$ (Goal)	0	0	0	0

```

1 # R shape: (48, 4)
2 R = np.full((48, 4), -1.0)
3
4 for s in range(48):
5     row, col = state_to_rc(s)
6     for a in range(4):
7         dr, dc = ACTION_DELTAS[a]
8         nr = max(0, min(3, row+dr))
9         nc = max(0, min(11, col+dc))
10        ns = rc_to_state(nr, nc)
11
12        if ns in CLIFF_STATES:
13            R[s, a] = -100.0
14        elif ns == GOAL_STATE:
15            R[s, a] = 0.0
16        # else: stays -1.0

```

Note

The penalty is assigned to the *action* that leads to the cliff, not to being in the cliff state – consistent with standard tabular-MDP conventions.

Deterministic dynamics

$$\mathcal{T}(s, a, s') = \begin{cases} 1 & s' = f(s, a) \\ 0 & \text{otherwise} \end{cases}$$

where the next-state function f is:

$$f(s, a) = \begin{cases} S_{\text{start}} & \text{wall-bounce} \in \mathcal{C} \\ s & s = S_G \text{ (absorbing)} \\ \text{wall-bounce}(s, a) & \text{else} \end{cases}$$

Normalisation (always verified)

$$\sum_{s' \in \mathcal{S}} \mathcal{T}(s, a, s') = 1 \quad \forall s, a$$

```
1 assert T.sum(axis=2).min() == 1.0
```

```
1 # T shape: (48, 4, 48)
2 T = np.zeros((48, 4, 48))
3
4 for s in range(48):
5     if is_terminal(s): # goal
6         T[s, :, s] = 1.0 # absorb
7         continue
8     for a in range(4):
9         ...
10        if ns in CLIFF_STATES:
11            ns = START_STATE # reset
12        T[s, a, ns] = 1.0
```

Memory

$$\mathcal{T} \in \mathbb{R}^{48 \times 4 \times 48} = 9,216 \text{ entries}$$

Each slice $\mathcal{T}[s, a, \cdot]$ is a one-hot vector (deterministic), so a sparse representation is equally valid for large state spaces.

State-value function

For policy π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

Bellman optimality equation

$$V^*(s) = \max_{a \in \mathcal{A}} \left[\mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{T}(s, a, s') V^*(s') \right]$$

Q-function

$$Q^*(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{T}(s, a, s') V^*(s')$$

$$V^*(s) = \max_a Q^*(s, a)$$

Value iteration update

Initialise $V_0(s) = 0$. Repeat:

$$V_{k+1}(s) \leftarrow \max_a \left[\mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{T}(s, a, s') V_k(s') \right]$$

Convergence (contraction mapping)

Under the ℓ_∞ norm:

$$\|V_{k+1} - V^*\|_\infty \leq \gamma \|V_k - V^*\|_\infty$$

For $\gamma < 1$ the map is a contraction (Banach fixed-point theorem) $\Rightarrow V_k \rightarrow V^*$.

Stop when $\max_s |V_{k+1}(s) - V_k(s)| < \theta$.


```

1 V = np.zeros(NUM_STATES) # V_0 = 0
2
3 for i in range(1, max_iters+1):
4     delta = 0.0
5     V_new = np.zeros(NUM_STATES)
6
7     for s in range(NUM_STATES):
8         if is_terminal(s):
9             V_new[s] = 0.0 # V*(G)=0
10            continue
11
12            # Q = R[s] + gamma * T[s] @ V
13            # Q ∈ ℝ⁴, one entry per action
14            Q = (REWARDS[s]
15                 + gamma*(TRANSITIONS[s] @ V))
16
17            V_new[s] = Q.max() # V_{k+1}(s) = max_a Q(s, a)
18            delta = max(delta,
19                        abs(V_new[s] - V[s]))
20
21 V = V_new
22 if delta < theta: # ||ΔV||_∞ < θ
23     break

```

Matrix form of the Q -update

$\text{TRANSITIONS}[s]$ has shape $(4, 48)$:

$$\underbrace{\mathbf{Q}}_{4 \times 1} = \underbrace{\mathbf{R}[s, \cdot]}_{4 \times 1} + \gamma \underbrace{\mathbf{T}[s, \cdot, \cdot]}_{4 \times 48} \underbrace{\mathbf{V}}_{48 \times 1}$$

A single $@$ (matmul) replaces the inner $\sum_{s'}$ loop over all 48 states.

Policy extraction

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

```

1 def extract_policy(V, gamma):
2     Q = (REWARDS
3          + gamma*(TRANSITIONS @ V))
4     # Q shape: (48, 4)
5     return Q.argmax(axis=1)

```

Error bound after k sweeps

$$\|V_k - V^*\|_\infty \leq \frac{\gamma^k}{1 - \gamma} \|V_1 - V_0\|_\infty$$

For $\gamma = 0.99$, $\theta = 10^{-8}$:

$$k \approx \frac{\ln(\theta(1 - \gamma)/\|V_1 - V_0\|_\infty)}{\ln \gamma} \approx 3,800$$

Time complexity per sweep

$$O(|S|^2 \cdot |A|) = O(48^2 \times 4) = O(9,216)$$

Total: $O(k \cdot |S|^2 \cdot |A|)$.

Why $\gamma < 1$ matters

- $\gamma = 1$: contraction fails; the algorithm may diverge for undiscounted problems with cycles.
- $\gamma = 0.99$: path of 13 steps discounts minimally – near-undiscounted in practice.

Expected output ($\gamma = 0.99$)

State	$V^*(s)$
Start (3, 0)	≈ -13.2
Top-left (0, 0)	≈ -13.0
Goal (3, 11)	0.0

Optimal path: Up, Right $\times$ 11, Down.

Mathematical model

At each time step t :

- 1 Observe $S_t = s$
- 2 Select $A_t = \pi^*(s)$
- 3 Receive $R_{t+1} = \mathcal{R}(s, A_t)$
- 4 Transition: $S_{t+1} = \arg \max_{s'} \mathcal{T}(S_t, A_t, s')$

Episode return (undiscounted)

$$G_0 = \sum_{t=0}^{T-1} R_{t+1}$$

Optimal 13-step path:

$$G_0 = (-1) \times 13 = -13$$

```
1 def step(state, action):
2     reward = REWARDS[state, action]
3     # one-hot row: argmax gives s'
4     next_s = int(np.argmax(
5         TRANSITIONS[state, action]
6     ))
7     return next_s, reward
8
9 def run_episode(policy,
10                 max_steps=200,
11                 verbose=True):
12     s = START_STATE
13     total_r = 0.0
14     for t in range(max_steps):
15         if is_terminal(s):
16             break
17         a = policy[s]
18         s, r = step(s, a)
19         total_r += r
20     return total_r
```

Termination

The goal is the only terminal state. Cliff cells reset $S_t \leftarrow S_{\text{start}}$; the episode *continues*.

Trajectory as a random variable

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T)$$

$$p(\tau) = \prod_{t=0}^{T-1} \underbrace{\mathcal{T}(s_t, a_t, s_{t+1})}_{\in \{0,1\}} \cdot \pi(a_t | s_t)$$

For our deterministic π and $\mathcal{T}$: $p(\tau^*) = 1$.

Multiple-episode statistics

Run N episodes, compute:

$$\bar{G} = \frac{1}{N} \sum_{i=1}^N G_0^{(i)}, \quad \sigma_G = \sqrt{\frac{1}{N} \sum_i (G_0^{(i)} - \bar{G})^2}$$

Deterministic optimal policy: $\bar{G} = -13$, $\sigma_G = 0$.

Verbose step trace (sample)

t	s	a	R_{t+1}
0	36 (3, 0)	Up	-1
1	24 (2, 0)	Right	-1
2	25 (2, 1)	Right	-1
⋮	⋮	⋮	⋮
12	35 (2, 11)	Down	-1
-	47 (3, 11)	Goal!	-

$$G_0 = -13$$

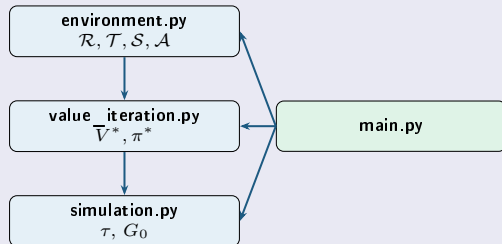
run_multiple_episodes()

Calls `run_episode()` N times silently and prints $\bar{G}$ and σ_G .

main.py – Orchestration

```
1 # 1. Environment built on import
2 from environment import (
3     REWARDS, TRANSITIONS, ...)
4
5 # 2. Value Iteration
6 from value_iteration import (
7     value_iteration)
8 V, policy, iters = value_iteration(
9     gamma=0.99,
10    theta=1e-8)
11
12 # 3. Simulation
13 from simulation import (
14     run_episode,
15     run_multiple_episodes)
16
17 run_episode(policy, verbose=True)
18 run_multiple_episodes(policy, n=10)
```

Data flow



No extra dependencies

pip install numpy is sufficient.

Summary – Mathematical Role of Each File

File	Math object	Key formula
environment.py	$\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}$	$s = 12r + c; \quad \mathcal{R}(s, a) \in \{-100, -1, 0\};$ $\sum_{s'} \mathcal{T}(s, a, s') = 1$
value_iteration.py	V^*, π^*, Q^*	$V_{k+1}(s) = \max_a [\mathcal{R}(s, a) + \gamma \mathcal{T}[s] \mathbf{V}_k];$ $\pi^*(s) = \arg \max_a Q^*$
simulation.py	τ, G_0	$S_{t+1} = \arg \max_{s'} \mathcal{T}(S_t, A_t, s'); \quad G_0 = \sum_t R_{t+1}$
main.py	Orchestration	Imports and calls all three modules

Key parameters

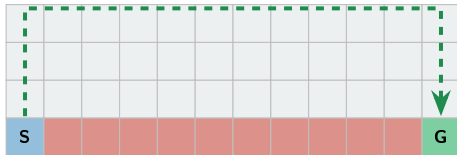
- $|\mathcal{S}| = 48, |\mathcal{A}| = 4, \gamma = 0.99, \theta = 10^{-8}$
- Optimal return: $G_0^* = -13$
- Optimal path: 13 steps

Theorems applied

- Bellman optimality principle
- Banach contraction mapping
- Policy improvement theorem

Thank you!

Cliff Walking MDP – complete.



Optimal path: Up $\rightarrow$ Right $\times 11 \rightarrow$ Down $G_0^* = -13$